

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501
NAME: M.NITHYASHREENARAYANI
REGNO: 192521416

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index	BL4 – Analyze	5

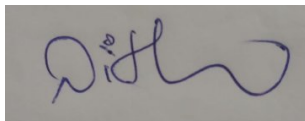
	structure.		
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I M.NITHYASHREENARAYANI / 192521416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of	25%	Applies advanced	Applies appropriate	Limited application	Incorrect or no

Engineering Concepts		and relevant concepts effectively to solve the problem	concepts with minor errors	of concepts	application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

File Organization and Indexing in DBMS

Assessment Answer Sheet — 10 Applied/Analytical Questions

Topics: File Organization, Indexing, Hashing, B-Tree/B+ Tree Structures

Bloom's Taxonomy Levels Covered: BL3 (Apply), BL4 (Analyze)

Q1. E-commerce company — file organization strategy for 10 million product records (with cost analysis)

Bloom's Level 4 — Analyze

1. Understanding access requirements

An e-commerce catalog is read-heavy and query-diverse: customers search by **ProductID** (point lookup at checkout/cart), by **category** / **price range** (range queries), and by **keyword** (full-text search). Writes (new listings, stock/price updates) are frequent but far less frequent than reads (typical read:write ratio 20:1 or higher in retail catalogs).

2. Candidate organizations

Organization	Search (point)	Search (range)	Insert	Delete	Suitability
Heap (unordered)	$O(n)$	$O(n)$	$O(1)$	$O(n)$	Poor — no query support
Sorted (sequential) file	$O(\log_2 n)$	$O(\log_2 n + k)$	$O(n)$	$O(n)$	Good for range, costly writes
Hashed file	$O(1)$ avg	$O(n)$ (no order)	$O(1)$ avg	$O(1)$ avg	Great for ProductID lookup only
Indexed-Sequential (ISAM)	$O(\log_2 n)$	$O(\log_2 n + k)$	Degrades (overflow)	$O(\log_2 n)$	Static catalogs only
B+ Tree clustered + hash secondary	$O(\log n)$	$O(\log n + k)$	$O(\log n)$	$O(\log n)$	Best overall fit

3. Recommended strategy

- **Primary organization:** a clustered **B+ Tree file organization** on **ProductID** (the primary key). Leaf nodes hold the actual rows (or row pointers) and are linked, giving both fast point lookup and efficient range/sequential scans for listing pages.
- **Secondary hashed index** on **ProductID** is unnecessary once B+ Tree is clustering key access, but a **hash index** is added on high-selectivity equality-only attributes such as **SKU** or **Barcode** used at POS scanning, since $O(1)$ beats $O(\log n)$ for pure equality lookup at scale.
- **Secondary B+ Tree indexes** on **CategoryID** and **Price** to support faceted browsing/range filters.
- **Inverted index** (or delegate to a search engine such as Elasticsearch/Solr) for free-text product search — this is outside classical relational file organization but is standard practice in production e-commerce systems.
- **Partitioning/sharding** by category or hash of ProductID once volume exceeds a single node's practical size, to keep each B+ Tree shallow (typically 3–4 levels even at 10M+ rows).

4. Cost analysis

10,000,000 records, average record size 200 bytes, block size 4 KB (4096 bytes), block pointer 6 bytes, key size (ProductID) 10 bytes.

1. **Blocking factor (data):** $bfr = \lfloor 4096 / 200 \rfloor = 20$ records/block.
2. **Data blocks required:** $b = \lceil 10,000,000 / 20 \rceil = 500,000$ blocks.
3. **Heap file search cost:** average $b/2 = 250,000$ block I/Os for an unindexed equality search — completely impractical for a live storefront.
4. **Sorted file (binary search):** $\log_2(500,000) \approx 19$ block I/Os — fast reads, but every insert/delete requires re-sorting or shifting, costing up to 500,000 I/Os in the worst case — unacceptable for a catalog with constant stock/price updates.
5. **B+ Tree index cost:** index entry size = 10 + 6 = 16 bytes → fan-out order $p \approx \lfloor 4096/16 \rfloor = 256$. Levels needed $\approx \log_{256}(10,000,000) \approx 3$ levels. Point search cost $\approx 3\text{--}4$ I/Os (root and internal nodes are cached in RAM in practice, so effective cost is often 1 I/O for the leaf).
6. **Hash index cost:** $\sim 1.0\text{--}1.2$ I/Os average for equality lookup with a well-tuned bucket load factor (~ 0.8).

Cost comparison summary (10M rows, per-query disk I/O)

Heap scan:	$\sim 250,000$ I/Os	(unusable)
Sorted binary search:	~ 19 I/Os	(read-fast, write-slow)
B+ Tree index:	$\sim 3\text{--}4$ I/Os	(read+write balanced)
Hash index:	$\sim 1\text{--}1.2$ I/Os	(equality only, no ordering)

5. Justification

A clustered **B+ Tree** organization is recommended as the primary structure because it delivers logarithmic cost for both point and range queries, remains balanced under heavy insert/delete traffic (unlike sorted/ISAM files, which need periodic reorganization), and its linked-leaf structure directly serves paginated product listings. A **hash index** is layered on top for pure equality lookups (SKU/barcode scanning) where $O(1)$ access materially improves latency at checkout. This hybrid approach keeps storage overhead modest (index overhead is typically 5–10% of data size) while keeping every supported query pattern within a handful of disk I/Os even at 10 million rows — a cost profile the heap or plain sorted-file alternatives cannot match.

Q2. Banking system — indexing strategy for fast retrieval by account number and date range

Bloom's Level 4 — Analyze

1. Query pattern analysis

- **Point query:** “Get all transactions for account number X” — equality on **AccountNo**.
- **Range query:** “Get transactions for account X between date D1 and D2” — equality + range, a **composite** predicate.
- Banking data is append-mostly (transactions are rarely updated/deleted once posted) and volumes are extremely high, so the index must sustain fast writes without frequent rebalancing storms.

2. Design: two-level combined indexing

(a) **Clustering primary index — B+ Tree on AccountNo:** transaction records are physically clustered (or index-organized) by **AccountNo**, since most queries begin with “all transactions for this account.” This groups every transaction for a customer into contiguous blocks, minimizing I/O for the common case.

(b) Composite (concatenated) secondary index — B+ Tree on (AccountNo, TransactionDate): within each account's cluster, a composite index keyed on (AccountNo, Date) allows the date range to be resolved as a contiguous scan of the leaf level once the account prefix is located — this is the standard way relational engines (Oracle, PostgreSQL, SQL Server) satisfy “equality-then-range” predicates efficiently.

(c) Optional non-clustering index on TransactionDate alone: for bank-wide reporting queries (“all transactions on date D across all accounts”, used for reconciliation/audit), a separate non-clustering B+ Tree index on TransactionDate is maintained.

3. Why B+ Tree, not B-Tree or hashing

Structure	Point query (AccountNo)	Range query (Date)	Verdict
Hash index	$O(1)$ — excellent	Not supported (no order)	Reject — fails range needs
B-Tree (data in internal nodes)	$O(\log n)$	Range scan needs in-order traversal with backtracking	Inefficient for ranges
B+ Tree (data only in linked leaves)	$O(\log n)$	$O(\log n + k)$ via leaf-chain scan	Best fit

4. Composite index structure (illustration)

Composite key entries are ordered first by AccountNo, then by Date, e.g.:

-
-
-
-

A query for account AC1001 between 2026-02-01 and 2026-03-01 descends the B+ Tree once to locate the (AC1001, 2026-02-01) boundary, then follows leaf-node sibling pointers sequentially until the date exceeds 2026-03-01 — no re-traversal from the root is needed per row.

5. Additional considerations

- **Write-ahead logging & index locking:** banking systems need indexes that support concurrent insert-heavy workloads with row-level locking (B+ Trees with high fan-out reduce lock contention vs. hash buckets).
- **Partition-by-date:** older transactions can be range-partitioned by month/year (partition pruning), shrinking the effective B+ Tree height for recent-data queries which dominate OLTP traffic.
- **ACID compliance:** B+ Tree index updates are done within the same transaction as the data write, ensuring the index never diverges from committed data — critical for financial correctness.

6. Conclusion

A clustering B+ Tree on AccountNo combined with a composite (AccountNo, Date) secondary B+ Tree index, supplemented by date-based range partitioning, gives the banking system logarithmic-cost point lookups and efficient sequential range scans while remaining safe for high-frequency concurrent inserts — properties that hashing (no ordering) and plain B-Trees (no leaf-chaining) cannot provide together.

Q3. Healthcare system — multi-level indexing for PatientID and doctor name access

Bloom's Level 4 — Analyze

1. Requirement

Patient records must be retrievable both by **PatientID** (unique, primary key, used at registration/billing) and by **DoctorName** (non-unique, used when a physician pulls up their patient list). This is a classic case for combining a **primary index** with a **multi-level secondary index**.

2. Design

Layer 1 — Primary (clustering) index on PatientID: patient records are stored physically ordered by **PatientID**, with a sparse primary index (one entry per block) pointing to the first record of each block. Because PatientID is unique, a dense index isn't required, saving index space.

Layer 2 — Secondary index on DoctorName: since records are ordered by PatientID (not DoctorName), a **dense secondary index** is required — one entry per record, since DoctorName is non-unique and unordered relative to physical storage. Each entry maps **DoctorName** → **record pointer** (or, better, to a small bucket of pointers, since one doctor has many patients).

Layer 3 — Multi-level index-on-index: because both PatientID and DoctorName secondary indexes can themselves grow too large to fit in a single block once patient volume is large, each is turned into a **multi-level (B+ Tree) index**: the first-level index (of index entries) is itself indexed by a second-level sparse index, and so on until the top level fits in one block — this is exactly the B+ Tree construction principle.

3. Structural diagram (conceptual)

Level	PatientID path	DoctorName path
Data file	Records physically ordered by PatientID	Same physical file (unordered w.r.t. DoctorName)
Level-1 index	Sparse primary index (1 entry/block) on PatientID	Dense secondary index (1 entry/record) on DoctorName, entries sorted alphabetically
Level-2 index	Sparse index over Level-1 (fits root in memory)	Sparse index over Level-1 doctor-name entries
Root	Single block, cached in memory	Single block, cached in memory

4. Why a secondary index on DoctorName must be dense + multi-level

- DoctorName is not the physical ordering key, so records for one doctor are scattered across many blocks — a sparse index cannot skip to the right block reliably; every matching record needs its own index entry (dense).
- A doctor with hundreds of patients produces many repeated DoctorName entries; grouping these under a single key with a list of pointers (an “index with bucket of pointers”) avoids duplicate key storage and speeds up “all patients of Dr. X” queries.
- As hospital records scale into millions, even the dense index outgrows one block, so it is organized as a multi-level B+ Tree so lookups remain $O(\log n)$ instead of degrading to a linear scan of the index itself.

5. Query walk-throughs

7. **Lookup by PatientID (e.g., billing):** traverse the primary B+ Tree — typically 2–3 I/Os — directly to the block containing the record.
8. **Lookup by DoctorName (e.g., “Dr. Meera's patient list today”):** traverse the secondary multi-level B+ Tree on DoctorName to the leaf level, retrieve the bucket of record pointers for that doctor, then fetch each referenced record — cost is roughly $O(\log n + k)$ where k is the doctor's patient count.

6. Conclusion

A **sparse, multi-level primary B+ Tree index on PatientID** combined with a **dense, multi-level secondary B+ Tree index on DoctorName** (using pointer buckets for repeated names) satisfies both access patterns with logarithmic-cost lookups, while keeping index storage overhead proportional to record count rather than to a full linear scan.

Q4. Logistics company — B-Tree vs B+ Tree for dynamic shipment record insertions/deletions

Bloom's Level 4 — Analyze

1. Workload characteristics

A logistics tracking system continuously creates shipment records (new pickups), updates their status, and eventually purges delivered/expired records — a workload dominated by **frequent, dynamic insertions and deletions**, plus range queries such as “all shipments created this week” or “all shipments in transit sorted by ETA.”

2. Structural comparison

Feature	B-Tree	B+ Tree
Data storage location	Data (or pointers to data) stored in internal AND leaf nodes	Data pointers stored ONLY in leaf nodes
Key redundancy	No duplication — each key appears once	Keys duplicated: leaf level repeats keys from upper levels
Leaf-node linking	Leaves are not linked to each other	Leaves linked in a sequential list (linked list)
Range query support	Requires in-order traversal with backtracking through internal nodes	Direct sequential scan across linked leaves — no backtracking
Fan-out for a given node size	Lower (internal nodes also store data/pointers)	Higher (internal nodes store keys only) → shallower tree
Insert/Delete cost	$O(\log n)$, but rebalancing can touch data-carrying internal nodes, which is costlier	$O(\log n)$; restructuring is confined mostly to leaf level + occasional internal split/merge
Point-query cost	Can be faster — a match may be found at an internal node before reaching a leaf	Always must reach a leaf, even for a key equal to an internal separator

3. Analysis for the logistics scenario

9. **Frequent inserts (new shipments):** B+ Tree's higher fan-out (internal nodes hold only keys, not full records) means fewer levels for the same number of shipment records, so each insert triggers fewer node splits and touches fewer disk blocks.
10. **Frequent deletes (delivered/expired shipments purged in batches):** because B+ Tree data pointers live only at the leaf level, deletion logic is simpler and localized — removing a key never requires patching an internal separator that also “was” a data pointer, which happens in B-Trees when the deleted key existed in an internal node.
11. **Range/sequential queries (“shipments in transit by ETA”, “daily manifest exports”):** the linked-leaf structure of a B+ Tree allows an efficient left-to-right scan without repeated root-to-leaf traversals — essential for generating sorted manifests and dashboards.
12. **Bulk loading / reporting:** nightly batch jobs that scan all shipments benefit directly from B+ Tree's sequential leaf-chain, whereas a B-Tree scan must perform a full in-order tree traversal.

4. Verdict

Recommendation

B+ Tree is more suitable for the logistics shipment-tracking system.
 Reasons: higher fan-out → shallower tree → cheaper inserts/deletes;
 linked leaves → efficient range scans for ETA/manifest queries;
 simpler, more localized rebalancing on delete since data lives only at leaves.

B-Trees remain preferable only in niche cases where point-query latency must be minimized and range scans are rare (e.g., certain in-memory index structures), which does not match a logistics workload's mix of high insert/delete churn and frequent range-based reporting. Virtually every production RDBMS (MySQL/InnoDB, PostgreSQL, Oracle, SQL Server) therefore uses B+ Tree variants for exactly this class of problem.

Q5. HR system with 5000 employees — static vs. extendible hashing

Bloom's Level 3 — Apply

1. Scenario parameters

Assume: $N = 5000$ employee records, average record size **100 bytes**, block size **4096 bytes** → blocking factor $bfr = \lfloor 4096/100 \rfloor = 40$ records/block. Employee counts in an HR system change slowly (hiring/attrition), so growth is modest and fairly predictable, but not perfectly static — the system should tolerate moderate fluctuation without a full reorganization.

2. Static hashing design

13. **Buckets needed:** $\lceil 5000/40 \rceil = 125$ buckets (round up for headroom, e.g., **150 buckets** at ~80% target load factor).
14. **Hash function:** $h(\text{EmployeeID}) = \text{EmployeeID} \bmod 150$, mapping each ID to one of 150 fixed buckets.
15. **Behavior:** $O(1)$ average insert/search/delete as long as the bucket count assumption holds and collisions stay within overflow chains.

3. Extendible hashing design

16. **Structure:** a directory of pointers (size 2^d , where d = global depth) to buckets, each bucket having its own local depth. Only the directory doubles when a bucket overflows and its local depth would exceed the global depth — existing buckets are not all rehashed, only the overflowing one is split.
17. **Behavior:** $O(1)$ average access; directory grows gracefully (doubling only when needed) as employee count increases, e.g., during a hiring wave or company expansion/merger.

4. Comparative evaluation

Criterion	Static Hashing	Extendible Hashing
Setup complexity	Simple — fixed bucket count chosen upfront	More complex — directory + local/global depth bookkeeping
Behavior on growth beyond capacity	Overflow chaining degrades performance; eventually needs full rehash/reorganization	Directory doubles and only the affected bucket splits — no full rehash
Behavior on shrinkage (attrition)	Buckets stay allocated, wasting space	Buckets/directory can merge and shrink
Disk I/O for directory	None needed	One extra I/O possible for directory access unless cached in memory
Best when	Data volume is stable/known and rarely changes size	Data volume is expected to grow/shrink dynamically and unpredictably

5. Recommendation for this HR system

With only 5000 employees and organizational growth that is typically slow, gradual, and roughly predictable (annual hiring cycles, not viral growth), **static hashing is the pragmatic choice**: it is simpler to implement and administer, has zero directory-lookup overhead, and periodic (e.g., yearly) bucket resizing during low-traffic maintenance windows is entirely acceptable at this scale.

However, if the company anticipates rapid expansion, frequent mergers/acquisitions, or wants to avoid any planned downtime for reorganization, **extendible hashing** is the more robust choice, since it adapts to volume changes incrementally without a disruptive full-file rehash — at the modest cost of one extra directory-indirection I/O per access.

Verdict for 5000-employee HR system

Default recommendation: Static hashing (125–150 buckets, ~80% load factor).
Switch to extendible hashing only if headcount volatility (mergers, rapid scaling, seasonal contract staff) is expected to be significant and frequent.

Q6. Airline reservation system — combined index for flight number (point) and date (range)

Bloom's Level 4 — Analyze

1. Query patterns

- **Point query:** “Show seat availability for flight AI-202” — equality on **FlightNumber**.
- **Range query:** “Show all flights from city A to city B between 10 Sept and 20 Sept” — range on **DepartureDate**, often combined with equality on route.
- Booking systems also need very high concurrent read throughput (seat searches vastly outnumber actual bookings), so index structures must be optimized primarily for fast, concurrent reads.

2. Proposed combined index structure

(a) Primary hash index on FlightNumber: since flight number lookups are pure equality and extremely frequent (every seat-search hits this), a hash index gives $O(1)$ average access — the fastest possible for this access pattern.

(b) Secondary B+ Tree composite index on (Route, DepartureDate): supports the “flights on route R between date D1 and D2” pattern efficiently: the tree descends to the route prefix, then scans the linked leaf chain across the date range — identical reasoning to the banking composite index in Q2.

(c) Bitmap index on Status (e.g., Scheduled/Delayed/Cancelled) and CabinClass: these are low-cardinality attributes frequently used as filters (“show only Economy, only Scheduled flights”); bitmap indexes AND/OR together extremely fast and are far more space-efficient than B+ Trees for such attributes.

3. Why this combination

Access need	Index used	Reason
Flight number exact match	Hash index	$O(1)$ equality lookup, no ordering needed
Route + date range	Composite B+ Tree (Route, Date)	Range scans via linked leaves; equality-then-range pattern
Status / cabin-class filters	Bitmap index	Low cardinality — compact, fast boolean combination
Seat map for a specific flight+date instance	Hash on (FlightNumber, Date)	Composite equality key, $O(1)$ access to that specific flight instance

4. Concurrency and consistency notes

- Seat inventory updates (booking/cancellation) must lock at the finest possible granularity (a single flight-instance row), so the hash index on FlightNumber (or FlightNumber+Date) is ideal since it isolates updates to one bucket without disturbing neighboring keys — unlike a B+ Tree page split, which can momentarily affect sibling entries.
- The B+ Tree route/date index is read-mostly (schedules change far less often than seat status), so it tolerates being slightly “heavier” to maintain in exchange for excellent range-scan performance.

5. Conclusion

The airline system is best served by a **hybrid index scheme**: a **hash index** for the high-frequency equality lookup on flight number/flight instance, layered with a **composite B+ Tree on (Route, DepartureDate)** for range-based schedule search, and **bitmap indexes** on low-cardinality filter attributes — matching each index technology to the access pattern it serves best rather than forcing one structure to do everything.

Q7. University student database — file organization and secondary indexing for department and CGPA range queries

Bloom's Level 4 — Analyze

1. Requirements

- Primary key access by **RollNo** / **StudentID** (registration, transcripts).
- Equality queries by **Department** (low cardinality — typically 10–30 departments) — “list all CSE students.”
- Range queries by **CGPA** (continuous numeric, high cardinality) — “students with CGPA between 8.0 and 9.0” for merit lists/scholarships.
- Combined queries are common: “CSE students with CGPA > 8.5.”

2. File organization choice

A **clustering index-sequential (B+ Tree) organization on StudentID** is used as the base file organization — records are physically ordered by StudentID, giving fast registrar-style lookups and efficient bulk operations like transcript generation, while still supporting insert/delete without full-file reorganization.

3. Secondary indexing design

(a) Bitmap index on Department: department has low cardinality and is frequently combined with other filters using AND/OR logic (e.g., “CSE OR ECE students”). Bitmap indexes are extremely compact here (one bit per student per department) and support very fast boolean combination — far more efficient than a B+ Tree for a handful of distinct values.

(b) B+ Tree secondary index on CGPA: CGPA is continuous/high-cardinality, so a B+ Tree is the natural fit — leaf nodes are linked in sorted CGPA order, letting a range query “ $8.0 \leq \text{CGPA} \leq 9.0$ ” be answered by one descent plus a sequential leaf scan.

(c) Combined (AND) query execution — “CSE AND CGPA > 8.5”: the query optimizer intersects the bitmap for Department = CSE with the set of record pointers returned by the CGPA B+ Tree range scan (converted to a temporary bitmap or sorted pointer list), returning only students satisfying both — this index intersection strategy avoids a full table scan.

4. Structure summary

Attribute	Cardinality	Query type	Index chosen	Reason
StudentID	Unique	Point (clustering key)	Primary B+ Tree (clustering)	Base physical ordering; supports point + sequential access
Department	Low (~20 values)	Equality, boolean combos	Bitmap index	Compact; fast AND/OR across few distinct values
CGPA	High (continuous)	Range	Secondary B+ Tree	Ordered leaf-chain

Attribute	Cardinality	Query type	Index chosen	Reason
				enables efficient range scan

5. Conclusion

A **B+ Tree clustering organization on StudentID**, paired with a **bitmap index on Department** and a **B+ Tree secondary index on CGPA**, lets the university database answer point, department-filtered, and CGPA-range queries — individually or combined — without resorting to full table scans, while keeping index storage overhead low by matching each index type to the cardinality of its attribute.

Q8. Disk block utilization — Heap, Sorted, and Hashed organizations (100,000 records × 50 bytes, 4KB blocks)

Bloom's Level 4 — Analyze

1. Given parameters

- Number of records, $r = 100,000$
- Record size, $R = 50$ bytes
- Block size, $B = 4096$ bytes (4 KB)

2. Base calculation — blocking factor

$bfr = \lfloor B / R \rfloor = \lfloor 4096 / 50 \rfloor = 81$ records per block ($81 \times 50 = 4050$ bytes used, 46 bytes unused per block — this 46-byte slack exists in ALL three organizations since it is a function of record/block size, not organization).

Minimum blocks required (no organizational overhead): $b = \lceil 100,000 / 81 \rceil = 1,235$ blocks.

3. Heap (unordered) file

- Records are packed in insertion order with no reserved slack beyond the base 46 bytes/block, so utilization is essentially the theoretical maximum.
- Blocks used $\approx 1,235$. Utilization = $4050/4096 = \sim 98.9\%$
- Deletions leave “holes” (tombstones) that are not reclaimed until compaction, so utilization degrades over time with heavy delete activity — but at creation/steady insert-only time, it is the most space-efficient.

4. Sorted (sequential) file

- Base layout is identical to heap (same $bfr = 81$) if built once and never modified: $\sim 1,235$ blocks, $\sim 98.9\%$ utilization
- In practice, sorted files reserve **free space per block (fill factor)** — commonly 70–80% — to absorb inserts without immediately cascading a full re-sort. At an 75% fill factor: usable records/block = $\lfloor 81 \times 0.75 \rfloor = 60$, requiring $b = \lceil 100,000/60 \rceil = 1,667$ blocks, i.e. $\sim 74\%$ utilization — the remaining 25% is deliberately reserved slack, not waste, since it avoids frequent full-file reorganization.

5. Hashed file

- Buckets are sized for a target load factor (commonly 80–85%) to minimize overflow-chain collisions.

- At an 80% load factor: usable records/block = $\lceil 81 \times 0.8 \rceil = 64$, requiring $b = \lceil 100,000/64 \rceil = 1,563$ blocks, i.e. $\sim 80\%$ utilization by design.
- If load factor is pushed higher (e.g., 95%) to save space, overflow chaining increases sharply, degrading average search cost — the classic space-vs-speed trade-off of hashing.

6. Consolidated comparison

Organization	Blocks required	Effective utilization	Notes
Heap	$\sim 1,235$	$\sim 98.9\%$	Max density; degrades with deletes (tombstones)
Sorted (75% fill factor)	$\sim 1,667$	$\sim 74\%$ (by design)	Reserved slack absorbs inserts without full re-sort
Hashed (80% load factor)	$\sim 1,563$	$\sim 80\%$ (by design)	Reserved slack limits overflow-chain collisions

7. Analysis

Raw block utilization is **highest for the heap file** because it has no organizational constraint requiring reserved space — but this is precisely why heap files are slow to search. Sorted and hashed files **deliberately trade utilization for performance**: their lower “designed” utilization (74–80% vs. heap’s $\sim 99\%$) is reserved headroom that keeps insert operations from immediately triggering expensive reorganization or long overflow chains. The correct conclusion is not that heap is “better” because it is denser, but that **utilization must be read alongside maintenance cost**: a heap file’s high density comes at the cost of $O(n)$ search, while sorted/hashed files intentionally under-fill blocks to keep $O(\log n)$ / $O(1)$ access cheap under ongoing inserts and deletes.

Q9. Social media platform — composite indexing for 500 million posts by user_id, timestamp, and hashtag

Bloom’s Level 4 — Analyze

1. Access pattern analysis

Query	Predicate shape	Cardinality	Frequency
“User’s own timeline”	<code>user_id = X, ORDER BY timestamp DESC</code>	<code>user_id</code> : very high cardinality	Extremely high (every profile view)
“Recent posts under #hashtag”	<code>hashtag = Y, ORDER BY timestamp DESC</code>	<code>hashtag</code> : skewed — few trending values dominate	Very high (trending pages)
“User’s posts with a hashtag”	<code>user_id = X AND hashtag = Y</code>	Combination, low result size	Moderate

2. Composite / multi-index design

(a) Composite B+ Tree on (user_id, timestamp): clustering or near-clustering index for the timeline feed — leaf entries sorted by timestamp within each user_id prefix let “latest N posts by this user” be answered as a bounded reverse leaf-chain scan, extremely cheap even at 500M rows since each user's own post count is a tiny fraction of the total.

(b) Inverted index on hashtag → posting list (post_id, timestamp): hashtags behave like search terms with highly skewed frequency (a trending tag may have millions of posts, most tags have a handful). An **inverted index** — the same structure used in text search engines — maps each hashtag to a sorted posting list of (timestamp, post_id), letting “recent posts for #hashtag” be served by scanning the head of that list; this scales far better than forcing a generic B+ Tree to handle such skew.

(c) Sharding/partitioning by user_id hash: at 500 million rows, no single index structure fits efficiently on one node. Horizontal partitioning by a hash of user_id distributes both storage and the (user_id, timestamp) index across shards, keeping each shard's B+ Tree shallow (a good practical target is a few million rows per shard, e.g. 3–4 B+ Tree levels).

(d) Secondary global index for hashtags across shards: because a single hashtag's posts are scattered across many user-sharded partitions, the hashtag inverted index typically needs to be maintained as a separate, globally-partitioned service (partitioned by hashtag itself, not by user), often backed by a search/indexing system rather than the primary transactional store — a widely used pattern (“scatter-gather avoidance”) in large-scale social platforms.

3. Why not one single composite index for everything

- A single index on (user_id, timestamp, hashtag) would only efficiently serve queries that filter on user_id first — it cannot serve “posts for #hashtag” without user_id, since that predicate isn't a leading prefix of the key.
- Similarly, (hashtag, timestamp, user_id) would fail to efficiently serve pure per-user timelines.
- Hashtag cardinality/skew (a handful of hashtags carrying enormous post volume) also behaves very differently from user_id's more even distribution, which is why an inverted-index/search-engine style structure suits hashtags better than a plain B+ Tree leaf entry per post.

4. Summary of the composite strategy

Component	Structure	Serves
(user_id, timestamp)	Composite B+ Tree, partitioned by user_id hash	Per-user timeline / “my posts”
hashtag → posting list	Inverted index (hashtag-partitioned)	“Posts under #tag” / trending pages
(user_id, hashtag)	Intersection of the two above at query time	“This user's posts with this hashtag”

5. Conclusion

At the 500-million-row scale, no single composite key satisfies all three access dimensions efficiently; the right design **decomposes the workload**: a **sharded composite B+ Tree on (user_id, timestamp)** for timelines, and a separate **hashtag-partitioned inverted index** for tag-based discovery, with query-time intersection for the rarer combined predicate — the same architectural pattern used by production social platforms.

Q10. Supermarket billing — Heap vs. Sorted vs. Hashed file performance for 1 million daily transactions

Bloom's Level 4 — Analyze

1. Workload profile

A billing/POS system is dominated by **inserts** (every checkout appends a new transaction record) with occasional lookups (returns/refunds by transaction ID or receipt number) and end-of-day range queries (“all transactions today” for reconciliation). Deletes are rare (transactions are typically archived, not deleted). This profile strongly favors write-optimized, append-friendly structures.

2. Complexity comparison

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$ — append at end	$O(n)$ — must maintain order (shift or gap-fill)	$O(1)$ avg — direct bucket write
Search (by key)	$O(n)$ — linear scan	$O(\log_2 n)$ — binary search	$O(1)$ avg — direct bucket access
Delete	$O(n)$ find + $O(1)$ mark	$O(n)$ — find + shift/compact	$O(1)$ avg — find bucket + remove
Range query (e.g., today's sales)	$O(n)$ — full scan	$O(\log_2 n + k)$ — fast if range key is sort key	$O(n)$ — no inherent order

3. Quantified estimate for 1,000,000 daily transactions

Assume record size 100 bytes, block size 4096 bytes → bfr = 40 records/block → b = 25,000 blocks/day.

Organization	Avg. search cost (I/Os)	Avg. insert cost (I/Os)	Delete cost (I/Os)
Heap	~12,500 (b/2)	1 (append)	~12,500 (find) + 1
Sorted	~15 ($\log_2 25,000$)	~12,500 (shift/insert-in-place) or deferred batch re-sort	~12,500 (find + shift)
Hashed	~1–1.2 (direct bucket)	~1–1.2 (direct bucket)	~1–1.2 (find bucket) + 1

4. Discussion

- **Heap file:** unbeatable for insert (simple append — exactly what happens at checkout, thousands of times per minute across registers) but every lookup (e.g., processing a return using the receipt number) costs a full scan, which is unacceptable at 1 million rows/day accumulated over time.
- **Sorted file:** excellent search cost, but maintaining physical order under near-continuous inserts is prohibitively expensive — real systems avoid keeping the *data* file sorted for exactly this reason, preferring to keep it heap-organized and add a sorted *index* instead.
- **Hashed file:** best all-around for this workload — $O(1)$ average insert AND $O(1)$ average point search by transaction ID/receipt number, which matches the two dominant operations (checkout write, return lookup) almost perfectly.

- **Range queries (daily sales reports):** none of the three natively excels here except the sorted file; the practical answer used in real POS systems is to keep the base table heap-organized (fast inserts) and add a **secondary B+ Tree index on transaction timestamp** purely for reporting — combining heap-file write speed with indexed range-read speed.

5. Recommendation

Recommended design for the supermarket billing system

Base file: Heap organization (or hash-organized by TransactionID) — maximizes insert throughput for the checkout-dominated workload.

Point lookup (returns/refunds): Hash index on TransactionID/ReceiptNo — O(1) average search without slowing down inserts.

Range/reporting (daily sales, audits): Secondary B+ Tree index on timestamp — supports fast 'today's transactions' style range scans without forcing the base file itself to stay physically sorted.

This layered approach — **heap/hash base file + B+ Tree secondary index** — outperforms using any single organization alone, because it matches the write-dominant, occasionally-range-queried nature of retail billing far better than a plain sorted file (write-costly) or a plain heap file (search-costly) used in isolation.